Taller de Programación sobre GPUs



Introducción a HPC y GPGPU

Universidad Nacional de La Plata



Facultad de Informática

Prof. Dr. Adrian Pousa



1. **Introducción al concepto de HPC**
2. **La GPU como arquitectura paralela**
   Mecanismos de control (Taxonomía de Flynn). Modelos de comunicación. Granularidad.
3. **La GPU y los algoritmos paralelos**
   Descomposición de problemas. Características de las aplicaciones. Herramientas de programación paralela
4. **Relación arquitectura paralela - programación paralela**
5. **Evolución HPC**
6. **Resumen de características**



1 Introducción al concepto de HPC

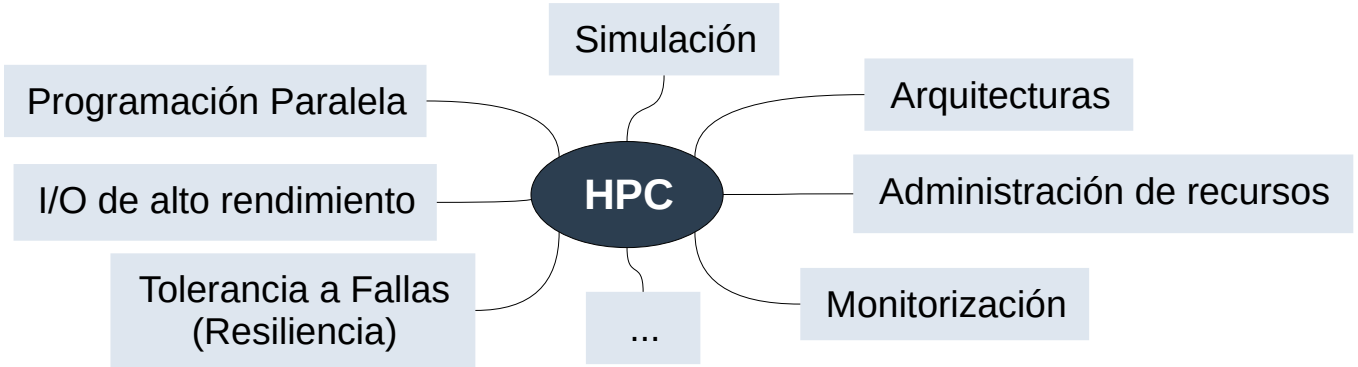
Introducción al concepto de HPC

High Performance Computing (HPC)

(Cómputo de altas prestaciones o computación de alto rendimiento)

HPC: Práctica de sumar potencia de cómputo con el fin de alcanzar mayor rendimiento en la ejecución de programas que resuelven problemas complejos o de gran tamaño de la ciencia, ingeniería o negocios.

- **HPC** abarca el estudio de distintas áreas:



Introducción al concepto de HPC
Sistema Paralelo

Sistema Paralelo: Sistema organizado de *hardware* y *software* paralelos que permite ejecutar múltiples tareas de manera simultánea. Su objetivo es mejorar el rendimiento respecto a un algoritmo secuencial, ya sea reduciendo el tiempo de ejecución, aumentando la eficiencia energética o logrando ambos beneficios.



- Un **Sistema Paralelo** está compuesto por **dos componentes** principales y un **objetivo** fundamental:
  - **Software Paralelo (Algoritmo Paralelo):** Conjunto de **hilos o procesos** que **cooperan** en la resolución de un problema.
  - **Hardware Paralelo (Arquitectura Paralela):** Conjunto de **unidades de procesamiento** que permiten la ejecución simultánea de los hilos o procesos del software paralelo.
  - **Objetivo:** Mejorar el rendimiento respecto a la ejecución secuencial del problema.



2

La GPU como arquitectura paralela

Mecanismos de control (Taxonomía de Flynn)

Modelos de comunicación

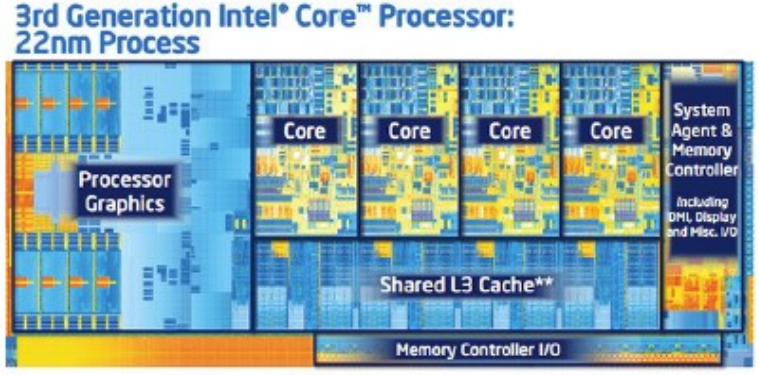
Granularidad

La GPU como arquitectura paralela

¿Qué es una GPU?

- **GPU** son las iniciales de ***G****raphics* ***P****rocessing* ***U****nit* (unidad de procesamiento gráfico)
- Es un coprocesador creado para procesamiento gráfico y liberar a la CPU del renderizado.
- Actualmente, se utiliza para procesamiento de propósito general (GPGPU) no gráfico.
- Generalmente, una GPU se conecta vía puerto PCI express (Nvidia y ATI-AMD) aunque los modelos de CPUs nuevos incorporan una GPU dentro del circuito integrado (Intel y AMD)





La GPU como arquitectura paralela
Clasificación

- Existen varias clasificaciones de arquitecturas paralelas.
- Para ubicar a las GPUs como arquitectura paralela vamos a enfocarnos sólo en algunas de estas clasificaciones:
  - Mecanismos de control (Taxonomía de Flynn)
  - Modelo de comunicación (relacionado con la memoria)
  - Granularidad (relacionado con el número y potencia de las unidad de procesamiento)



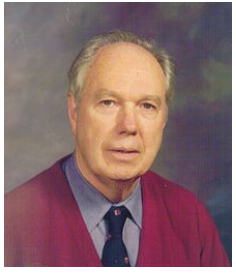
2

La GPU como arquitectura paralela

- *Mecanismos de control (Taxonomía de Flynn)*
- *Modelos de comunicación*
- *Granularidad*

Taxonomía de Flynn

- En 1972, Michael J. Flynn propone una clasificación de arquitecturas de cómputo*, conocida como Taxonomía de Flynn, basada en dos aspectos:
  - **El flujo de instrucciones concurrentes (control):** cantidad de flujos de control independientes que pueden ejecutarse en paralelo en el sistema. El flujo de instrucciones puede ser simple (sin paralelismo) o múltiple (con paralelismo).
  - **El flujo de datos:** datos sobre los cuales las instrucciones operan. Los flujos de datos pueden ser simples (sin paralelismo) o múltiples (con paralelismo).
- A partir de lo anterior, podemos clasificar las arquitecturas de cómputo en:

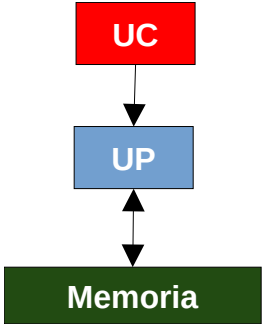


	Una Instrucción	Múltiples instrucciones
Un dato	**SISD**	**MISD**
Multiples datos	**SIMD**	**MIMD**

* “Some computer organizations and their effectiveness” https://ieeexplore.ieee.org/document/5009071

Taxonomía de Flynn
SISD

- **Single Instruction Single Data (SISD):** Un único flujo de instrucciones y un único flujo de datos.
  - Arquitectura correspondiente al modelo secuencial tradicional.
  - Ejemplo: Monoprocesadores (la mayoría de las PCs del siglo XX)

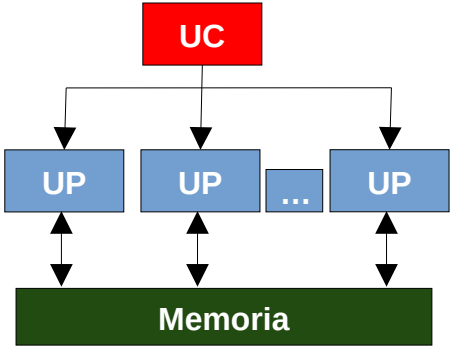


Flujo de ejecución:

1) La unidad de control (**CU**) envía instrucciones a la unidad de procesamiento (**UP**).
2) La **UP** lee datos de **memoria**, ejecuta la instrucción y escribe los resultados.
3) Todo ocurre de manera **secuencial**, sin paralelismo.

Taxonomía de Flynn
SIMD

- **Single Instruction Multiple Data (SIMD):** Un único flujo de instrucciones y múltiples flujos de datos.
  - Arquitectura donde la misma operación se ejecuta en paralelo sobre diferentes datos.
  - Ejemplos: **GPUs** y unidades vectoriales SSE o AVX

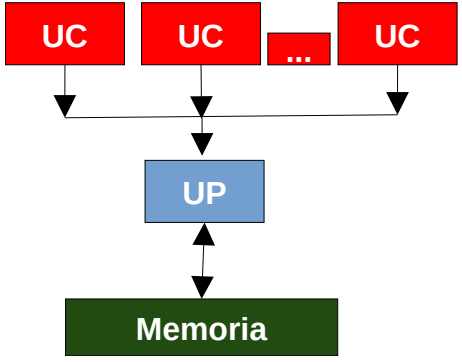


Flujo de ejecución:

1) La unidad de control (**CU**) envía la misma instrucción a todas las unidades de procesamiento (**UPs**).
2) Cada **UP** lee sus datos de la **memoria**, ejecuta la instrucción y escribe los resultados.
3) Todo ocurre en **paralelo** respecto a los datos, pero bajo un único flujo de instrucciones.

Taxonomía de Flynn
MISD

- **Multiple Instruction Single Data (MISD):** Múltiples flujos de instrucciones y un único flujo de datos.
  - Modelo de arquitectura poco común. Aplica distintas operaciones sobre los mismos datos.
  - Ejemplos asociados pero no reales: Sistemas redundantes tolerantes a fallos o Hacking de un mensaje codificado.

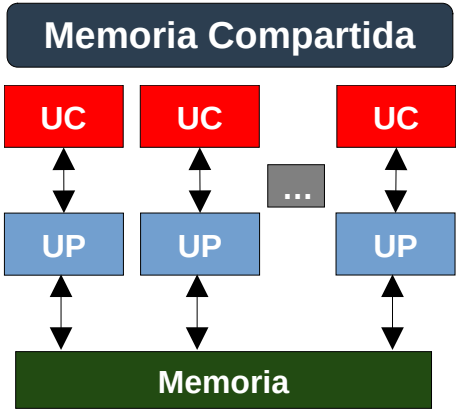


Flujo de ejecución:

1) Cada unidad de control (**CUs**) envía instrucciones a la única unidad de procesamiento (**UP**).
2) La **UP** lee el mismo flujo de datos de la **memoria**, ejecuta las instrucciones y escribe los resultados.
3) Todo ocurre en **paralelo** respecto a las instrucciones, pero sobre un único flujo de datos.

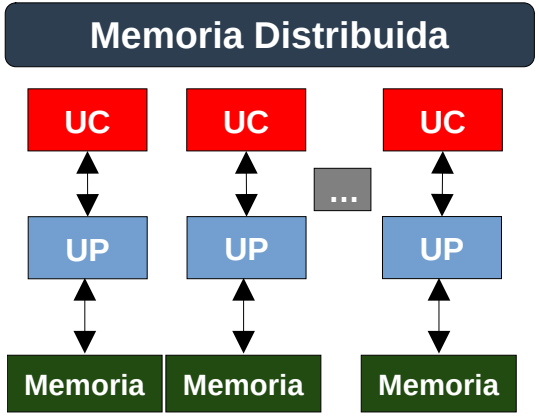
Taxonomía de Flynn
MIMD

- **Multiple Instruction Multiple Data (MIMD):** Múltiples flujos de instrucciones y múltiples flujos de datos.
  - Arquitectura donde cada unidad de procesamiento puede ejecutar su propio programa y trabajar sobre datos distintos.
  - Pueden ser de memoria compartida o distribuida.
  - Ejemplos: Multiprocesadores, Multicores, Clusters, Grid.



Flujo de ejecución:

1) Cada unidad de control (**CUs**) envía instrucciones de manera independiente a una unidad de procesamiento (**UP**).
2) Cada **UP** lee sus datos de la **memoria**, ejecuta la instrucción y escribe los resultados.
3) Todo ocurre en **paralelo** respecto a las instrucciones y a los datos.





2

La GPU como arquitectura paralela

Mecanismos de control (Taxonomía de Flynn)

Modelos de comunicación

Granularidad

Modelos de comunicación

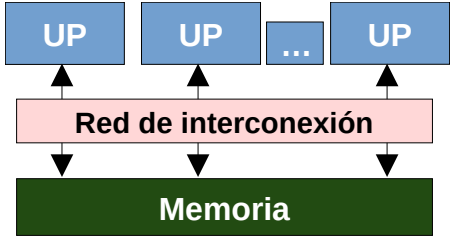
- De acuerdo con el modelo de comunicación entre tareas, las arquitecturas paralelas pueden clasificarse en dos grupos:
  - Multiprocesadores de **memoria compartida**:
    - **UMA** (Uniform Memory Access)
    - **NUMA** (Non-Uniform Memory Access)
  - Multiprocesadores de **memoria distribuida**

- Es posible tener ambos modelos (**modelo híbrido**).

Modelos de comunicación
Multiprocesadores de memoria compartida

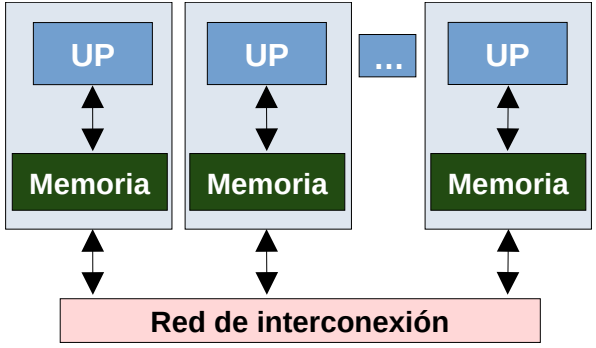
UMA (Uniform Memory Access)

- Unidades de procesamiento acceden a una única memoria compartida con un espacio de direcciones común.
- Tiempo de acceso a memoria uniforme para todas las unidades de procesamiento.
- Interconexión: bus o crossbar switch.



NUMA (Non-Uniform Memory Access)

- Memoria física distribuida entre unidades de procesamiento.
- Memoria local por cada unidad.
- Todas las memorias locales forman un único espacio de direcciones global.
- Tiempo de acceso a memoria no uniforme, accesos locales más rápidos que accesos remotos.

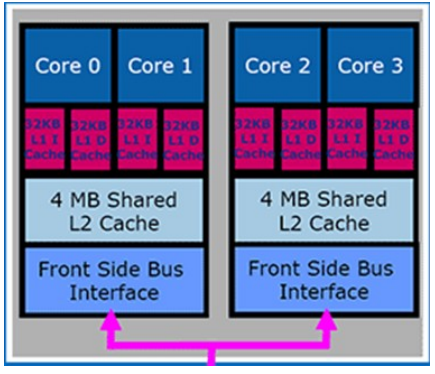


Limitaciones:

- Expansión limitada: mas unidades de procesamiento aumento de contención a memoria.
- Un fallo en una unidad de procesamiento afecta a todo el sistema.

La GPU como arquitectura paralela
Modelos de comunicación – Jerarquía de memoria

- En arquitecturas multicore es común tener jerarquías de memoria.
- Generalmente, dos o más niveles de cache (L1, L2, L3, L4)
- Según la arquitectura, los niveles 2, 3 y 4 suelen estar compartidos entre cores.

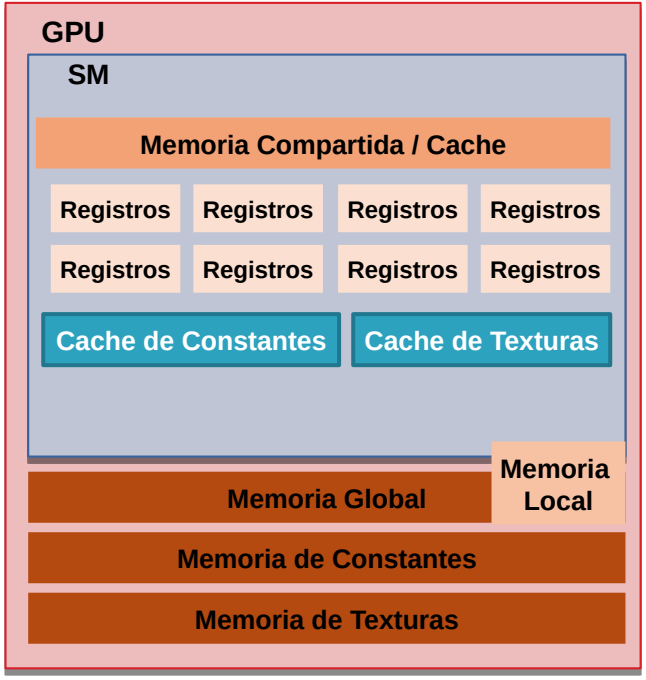






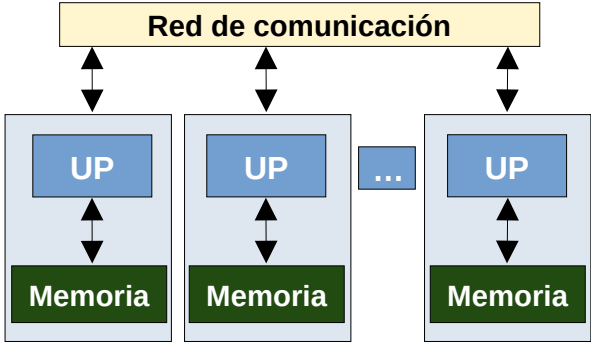
La GPU como arquitectura paralela
Modelos de comunicación – Jerarquía de memoria

- Las **GPUs** poseen una jerarquía de memoria más compleja.



Modelos de comunicación
Multiprocesadores de memoria distribuida

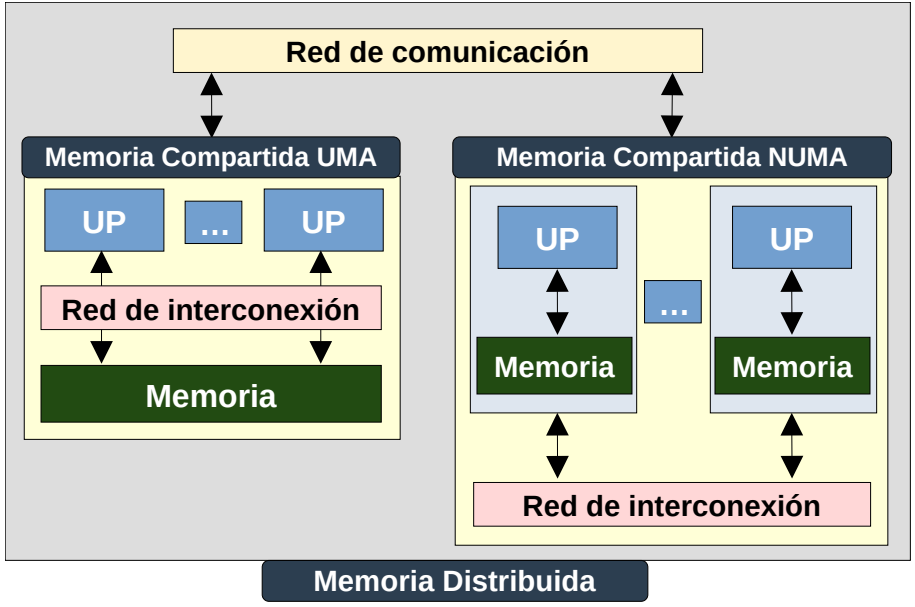
- Para simplificar, asumimos varias computadoras independientes con una unidad de procesamiento por computadora.
- Cada unidad de procesamiento posee su propia memoria local con su propio espacio de direcciones.
- Las unidades sólo tienen acceso directo a su memoria local y no acceden a memoria remota.
- La interacción entre unidades es sólo por pasaje de mensajes.



Ventajas:
- Expansible a bajo costo
- Fallos en una unidad no afecta a todo el sistema

Modelos de comunicación
Multiprocesadores híbridos

- Podemos componer el hardware paralelo en sistemas híbridos:
- Ventajas de ambos modelos:
  - Facilidad de acceso a memoria de los sistemas de memoria compartida.
  - Escalabilidad y tolerancia a fallos de los sistemas de memoria distribuida.





2 La GPU como arquitectura paralela

- *Mecanismos de control (Taxonomía de Flynn)*
- *Modelos de comunicación*
- *Granularidad*

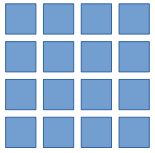
Granularidad

- **Granularidad (Hardware):** se refiere al número y la potencia de las unidades de procesamiento
- Las arquitecturas pueden ser:

Grano fino (fine-grained)

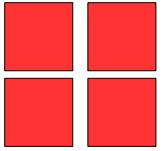
Muchas unidades de procesamiento poco potentes.

(GPUs)



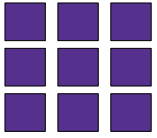
Grano grueso (coarse-grained)

Pocas unidades de procesamiento muy potentes.



Grano medio (medium-grained)

Equilibrio entre los anteriores.





3

La GPU y los algoritmos paralelos

Descomposición de problemas

Características de las aplicaciones

Herramientas de programación paralela



3

La GPU y los algoritmos paralelos

Descomposición de problemas

Características de las aplicaciones

Herramientas de programación paralela

La GPU y los algoritmos paralelos
Descomposición de problemas

- Para desarrollar un algoritmo paralelo el primer paso es descomponer el problema en sus partes concurrentes (**tareas**).
- Se trata de definir un gran número de pequeñas tareas (descomposición de grano fino), para brindar la mayor flexibilidad a los “potenciales” algoritmos paralelos
- Se ignoran cuestiones como el número de unidades de procesamiento y aspectos específico en la máquina de destino.
- La atención se centra en reconocer oportunidades de ejecución paralela.



En esta etapa:

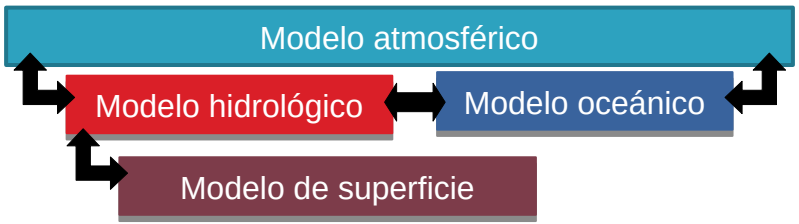
Las tareas **NO** son procesos o hilos

Tareas = cosas para hacer

La GPU y los algoritmos paralelos
Tipos de Descomposición

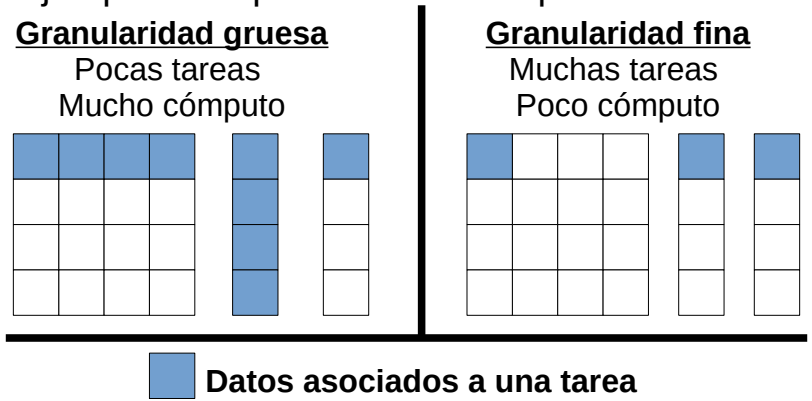
Descomposición Funcional

- Se enfoca principalmente en la funcionalidad.
- Divide el cómputo y luego asocia los datos.
- Generalmente tareas disjuntas (código diferente).
- Reduce la complejidad del diseño general: Modelos computacionales complejos pueden estructurarse como conjuntos de modelos más simples interconectados.
- Habituales en simulación:



Descomposición de datos o de dominio

- Se enfoca principalmente en los datos.
- Divide los datos y luego asocia el cómputo a tareas.
- Generalmente tareas de código igual o similar.
- Granularidad: volumen de trabajo de una tarea
- Ejemplo: Multiplicar una matriz por un vector



Descomposición Mixta

Se aplica una descomposición funcional macro con una descomposición de datos por cada función.

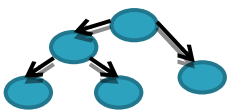
La GPU y los algoritmos paralelos
Descomposición de problemas

- La descomposición de datos o de dominio, también conocida como paralelismo de datos, se caracteriza por la ejecución simultanea de la misma operación sobre diferentes elementos de datos.
- Es más adecuado para arquitecturas de tipo SIMD.
- Los datos pueden tener estructuras:

Regulares (Arreglos, Matrices)



Irregulares (Grafos)



Las **GPUs** suelen adaptarse mejor al paralelismo de datos sobre estructuras regulares.



3

La GPU y los algoritmos paralelos

Descomposición de problemas

Características de las aplicaciones

Herramientas de programación paralela

Paralelismo explícito
Comportamiento de las aplicaciones

- Debemos considerar el comportamiento de las aplicaciones a paralelizar:
  - **Intensivas en cómputo** (CPU Bound): llevan al límite la capacidad de la CPU.
    - La mayoría de las aplicaciones de cómputo científico.
  - **Intensivas en memoria** (Memory Bound): llevan al límite el sistema de memoria.
    - Ordenación
    - Búsquedas
  - **Intensivas en Entrada-Salida** (E/S Bound): llevan al límite el sistema de E/S.
    - Simulación
    - Big data
  - **Híbridos:** La aplicación presenta distintas fases intensivas.

Paralelismo explícito
Comportamiento de las aplicaciones

- Distintos eventos relacionados al cómputo requieren de un tiempo específico (Latencias).

Las magnitudes suelen ser despreciables y no permiten observar las diferencias reales.

¿Qué ocurre si escalamos ese tiempo?





Evento	Tiempo Real	Tiempo Escalado
1 ciclo de CPU	0.3 ns	1 s
Acceso a cache de Nivel 1	0.9 ns	3 s
Acceso a cache de Nivel 2	2.8 ns	9 s
Acceso a cache de Nivel 3	12.9 ns	43 s
Acceso a RAM	120 ns	6 min
E/S a SSD	50-150 µs	2-6 días
E/S a HDD	1-10 ms	1-12 meses
Internet: San Francisco a Nueva York	40 ms	4 años
Internet: San Francisco a UK	81 ms	8 años
Internet: San Francisco a Australia	183 ms	19 años
Reboot de SO de VM	4 s	423 años
Reboot de SO físico	5 m	32 milenios



La GPU y los algoritmos paralelos
Características de las aplicaciones

- Las aplicaciones intensivas en CPU se benefician más de las GPUs.
  - Esto por las características de las GPU como arquitecturas y su modelo de programación.
- Las aplicaciones intensivas en memoria se benefician pero no de la misma forma.
- Las GPU no tienen posibilidad de hacer entrada-salida, por esto las aplicaciones intensivas en este aspecto requieren relacionarse fuertemente con la CPU.



3

La GPU y los algoritmos paralelos

Descomposición de problemas

Características de las aplicaciones

Herramientas de programación paralela

La GPU y los algoritmos paralelos
Modelos de programación paralela

- El desarrollo de aplicaciones paralelas se basa en dos modelos de programación:

Modelo de memoria compartida	Modelo de memoria distribuida
Procesos/Hilos se comunican mediante la memoria. Herramientas de desarrollo: Pthreads OpenMP **CUDA (Sobre GPUs)** Otros (OpenCL, Cilk, Sycl...)	**Procesos/Hilos se comunican mediante mensajes.** Herramientas de desarrollo: PVM (en desuso) MPI Otros (RMI, Sockets, RPC...)
Modelo híbrido	
 Combinación de herramientas de desarrollo de ambos modelos *(Ejemplo: MPI + OpenMP + CUDA)*	



4 Relación arquitectura paralela - programación paralela

Paralelismo explícito

Modelos de programación paralela – Modelos de arquitectura paralela



- Los modelos de Programación Paralela (**Software**) y los modelos de Arquitectura Paralela (**Hardware**) son complementarios. Sin embargo, el modelo de programación paralela elegido depende del modelo de arquitectura paralela disponible.



		Software		
		Memoria Compartida (Ej: OpenMP)	**Memoria Distribuida** (Ej: MPI)	**Híbrido** (EJ: MPI + OpenMP)
Hardware	**Memoria Compartida** (Ej: multicore)	**Trivial** (Ej: OpenMP sobre multicore)	**Posible** (Ej: MPI sobre multicore)	**Posible (extraño)** (Ej: MPI + OpenMP sobre multicore)
	Memoria Distribuida (Ej: cluster monocore[1])	**(NO ADECUADO)** Single System Image(SSI) **Overhead**	**Trivial** (Ej: MPI sobre cluster monocore)	**Posible (muy extraño)** (Ej: MPI + OpenMP sobre cluster monocore)
	Híbrido (Ej: cluster multicore)	**(NO ADECUADO)** Single System Image(SSI) **Overhead**	**Posible** (Ej: MPI sobre cluster multicore)	**Trivial** (Ej: MPI + OpenMP sobre cluster multicore)

[1]Actualmente, los clusters monocore cayeron en desuso pero se mencionan para ejemplificar.



5 Evolución HPC

Evolución HPC

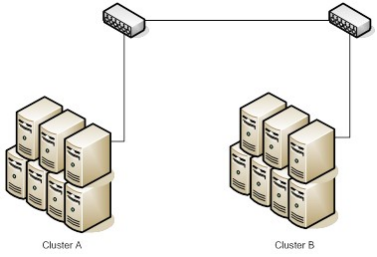


Primeras máquinas paralelas.
Muy costosas.

1950-1980

1989

Popularidad de los Clusters.
PCs + TCP/IP + PVM/MPI

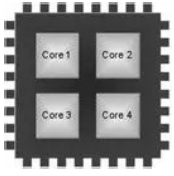


1995

Primeras “GPUs”
Renderizado.
No programables.

2005

Crisis del hardware.
Límite en la frecuencia (velocidad) de los procesadores.
Aparición de los Multicores (Pthreads/OpenMP)



2006

GPUs programables (Manycore).
Nvidia CUDA.
GPGPU.
AMD ATI.

2017

Tensor Cores.
GPGPU + IA.

Evolución HPC
Modelos Híbridos

- En la actualidad, las arquitecturas paralelas que se utilizan son sistemas híbridos, una combinación de arquitecturas de memoria compartida (Multiprocesadores y/o Multicores y/o Manycores-GPUs) y arquitecturas distribuidas (Clusters o Grid). En un futuro, posiblemente QPUs.





6 Resumen de características

Resumen de características

- Una **GPU** es una **Arquitectura de memoria compartida**.
- Una **GPU** sigue un **Modelo SIMD - SIMT**.
- Una **GPU** es una **Arquitectura de grano fino**.
- Una **GPU** se adapta mejor a **paralelismo de datos** (**regulares**)
- Una **GPU** es adecuada para **aplicaciones intensivas en CPU**.
- Es posible generar **Modelos híbridos** de gran potencia de cómputo:
  - Máquinas Multi-GPU
  - Maquinas Multicore/MultiGPU (OpenMP – Cuda, Pthreads – Cuda, etc)
  - Cluster de Multicore/MultiGPU (OpenMP – MPI – Cuda, Pthreads – MPI – Cuda, etc)

